

# Andrei Cirstescu — Systems Architecture

## Title

### Designing a Multi-Step AI Pipeline Microservice Architecture for Parallel Execution and Result Aggregation

## Context

MeetGeek's AI stack executes multi-step processing pipelines that include transcription, summarization, sentiment analysis, topic extraction, and action detection. These models have different computational demands and dependencies. A well-designed architecture can run independent models in parallel, merge results efficiently, and deliver faster insights to users while maintaining reliability.

## Objectives

- Architect a **multi-step AI pipeline** that supports **parallel execution** and **result aggregation**.
- Evaluate different **orchestration strategies** (e.g., DAG-based, event-driven, or queue-based pipelines).
- Ensure **fault tolerance, retry logic, and consistent state propagation**.
- Minimize total latency for complex AI workflows while maintaining modularity and scalability.

## Methodology

1. Model the current MeetGeek AI flow as a **directed acyclic graph (DAG)** of tasks.
2. Implement pipeline coordination using frameworks such as **Apache Flink, Celery, Ray**, or **Temporal.io**.
3. Enable **parallel execution** for independent AI services (e.g., summarization and topic extraction).
4. Implement a **result aggregation layer** that merges outputs into structured results.
5. Benchmark performance against sequential baselines using metrics like **end-to-end latency, throughput**, and **failure recovery time**.

## Expected Results

- A **modular and scalable architecture** for AI pipeline orchestration.

- Empirical proof of **reduced processing time** and improved fault tolerance.

## References

- Zaharia et al. (2016), "Apache Spark: Structured Streaming and the Future of Data Processing"
- Temporal.io (2024) Whitepaper, "Workflow Orchestration for AI Systems"
- MeetGeek AI Orchestration Design Notes (2025, internal)
- Dean & Ghemawat (2008), "MapReduce: Simplified Data Processing on Large Clusters"